

Python 网络数据挖掘

Mining the Web with Python

详细讲义

清华大学统计与数据科学系 | 俞声 | 2026.4.22

课程简介

机器学习的发展离不开大量数据的支撑。本课程介绍如何从互联网上构建数据集，主要涵盖两大技术路径：

- 网络爬虫（Web Scraping）：通过解析 HTML 结构自动提取网页信息
- 应用程序接口（API）：通过官方接口获取干净的结构化数据

常见的数据来源：Kaggle、Hugging Face、UC Irvine ML Repository 等平台已有大量公开数据集，但对于特定需求，往往需要自行采集。

第一部分：网络爬虫（Web Scraping）

1.1 什么是网络爬虫

网络爬虫（Web Scraping）是指通过解析 HTML 结构，自动从网页中提取信息的技术。

典型工作流程

- 下载 HTML 页面源代码
- 解析 HTML 的树状结构
- 提取所需目标数据

常用工具

- requests → 发送 HTTP 请求，获取页面内容
- Beautiful Soup → 解析 HTML 结构
- 浏览器 DevTools → 检查页面元素结构

"Tag soup" (标签汤) 是指结构混乱、标签嵌套错误的 HTML。BeautifulSoup 的名字正来源于此——它能处理各种混乱的 HTML。

1.2 实战示例：豆瓣电影 Top 250

目标网站: <https://movie.douban.com/top250>

该页面包含豆瓣评分最高的 250 部电影，分 10 页展示。我们的目标是抓取所有电影的标题、评分和简介。

获取 HTML 页面

最基本的做法是直接使用 requests 发送 GET 请求：

```
import requests

response = requests.get('https://movie.douban.com/top250')
html = response.text
```

上面的代码无法正常工作！响应状态码为 418，即服务器检测到了爬虫行为并拒绝了请求。

HTTP 状态码

服务器用状态码表示请求结果，需要了解常见状态码的含义：

状态码	含义	说明
200 OK	成功	GET 请求成功返回数据
301 Moved Permanently	重定向	资源已永久移动到新 URL
401 Unauthorized	未授权	需要提供身份验证信息
403 Forbidden	禁止访问	服务器拒绝执行请求
404 Not Found	未找到	请求的资源不存在
418 I'm a teapot	我是茶壶	非正式码，豆瓣用来表示检测到爬虫
429 Too Many Requests	请求过多	访问频率超出限制
502 Bad Gateway	网关错误	服务器端问题

418 I'm a teapot 并非正式 HTTP 标准，豆瓣借用此码拒绝爬虫请求，意为：我知道你在爬我。

解决方案：伪装成浏览器

在实际操作中，需要在请求头 (Headers) 中提供 User-Agent 信息，伪装成真实浏览器：

```
import requests

url = 'https://movie.douban.com/top250'
headers = {
    'User-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)\
                  AppleWebKit/537.36 (KHTML, like Gecko)\
                  Chrome/146.0.7680.80'
}

response = requests.get(url, headers=headers)
html = response.text

#python 代码一行没写完可以跟一个\
```

1.3 网页结构基础

网页的三大支柱

- HTML（超文本标记语言）：定义页面的结构和内容
- CSS（层叠样式表）：控制页面的视觉样式
- JavaScript：实现页面的动态功能和交互

HTML 结构

HTML 以树状结构（DOM 树）组织内容，每个元素用标签包裹：

```
<html>
  <head>
  </head>
  <body>
    <p>Here's a paragraph of text!</p>
    <p>Here's a second paragraph of text!</p>
  </body>
</html>
```

除了 HTML，另一个非常重要的超文本标记语言是 XML（eXtensible Markup Language，可扩展标记语言）。

特性	HTML (超文本标记语言)	XML (可扩展标记语言)
核心目标	呈现与布局 (网页内容)	存储与传输 (结构化数据)
标签定义	预定义的固定标签 (如 <p>, <div>)	自定义标签 (根据需求自己定义)
严格性	容错性强, 浏览器会尝试解析错误	语法极其严格, 格式错误直接报错
应用场景	网页骨架、前端开发	配置文件、API数据交换 (如 RSS)

常用 HTML 标签

标签	语法示例	用途说明
<div>	...	定义块级区域或分节
<p>	...	定义段落

标签	语法示例	用途说明
<a>	...	定义超链接
		嵌入图片
/	...	无序/有序列表
	...	列表项
<table>/<tr>/<td>	<table>...</table>	表格及行列
<h1>~<h6>	...	各级标题
	...	行内容器（无语义）

CSS 类名 (class) 与选择器

CSS 通过 class 属性为元素赋予样式，这也是我们定位目标元素的关键：

```
<!-- 方式一：内联样式（冗余） -->
<p style="color:red">Text 1</p>

<!-- 方式二：类名（推荐） -->
<p class="red-text">Text 1</p>
<p class="red-text">Text 2</p>

.red-text {
    color: red;
}
```

1.4 解析 HTML：Beautiful Soup

Python 网络爬虫主要依赖 BeautifulSoup 库来解析 HTML，配合 SelectorGadget（浏览器扩展）辅助识别元素规律。

定位目标元素有两种主流语法：

- **CSS Selector (CSS 选择器)**：利用 CSS 的类名、标签、层级等模式来匹配元素。语法简洁直观，初学者首选，也是 BeautifulSoup 的 .select() 方法所使用的方式。
- **XPath**：通过遍历 HTML 的树状结构来定位元素，表达能力更强，但语法相对复杂，适合进阶使用。

（课程建议初学者专注于 CSS Selector，掌握它就能应对绝大多数爬虫场景。）

CSS 选择器基础

BeautifulSoup 使用 CSS 选择器定位目标元素，以下是常用语法：

选择器	含义
div	选取所有 <div> 元素
.cls	选取所有 class 为 cls 的元素
#id	选取 id 为 id 的元素（通常唯一）
div p	选取 <div> 内所有 <p> 后代
div > p	选取 <div> 的直接子 <p>
div + p	选取紧跟在 <div> 后面的 <p>
.cls1.cls2	同时拥有 cls1 和 cls2 类的元素
[name="value"]	name 属性等于 value 的元素
.cls:first-child	有 cls 类且是父元素第一个子节点
.cls:nth-child(2)	有 cls 类且是父元素第二个子节点
.cls:first-of-type	父元素中 cls 类的第一个同类子节点
.cls:nth-of-type(2)	父元素中 cls 类的第二个同类子节点

Tag 对象操作方法

soup.select() 返回 Tag 对象列表，常用方法如下：

- `.text`（后者的简化版）或 `.get_text()`：获取标签内所有文本内容
- `.get('attribute')`：获取指定属性的值，例如 `.get('href')` 获取链接 URL
- `.select_one(selector)`：返回第一个匹配的 Tag 对象

提取豆瓣电影信息（基础版）

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html, 'html.parser')#用 html 解析器

# 提取中文标题
# .title:nth-child(1): 选取拥有 title 类、且是其父元素第 1 个子节点的元素（即中文标题，排在英文标题前面）
items = soup.select('.title:nth-child(1)')
title_cn = [item.text for item in items]

# 提取评分
# .rating_num: 选取所有拥有 rating_num 类的元素（即评分数字）
items = soup.select('.rating_num')
ratings = [item.text for item in items]

# 提取电影简介
# .quote span: 选取位于拥有 quote 类的元素内部的所有 <span> 子元素（即电影简短评语）
items = soup.select('.quote span')
quotes = [item.text for item in items]
```

```
# 组合成字典列表
data = []
for i in range(len(items)):
    data.append({
        "title_cn": title_cn[i],
        "rating": ratings[i],
        "quote": quotes[i]
    })
```

上述代码存在数据错位风险！当某些字段（如英文标题）缺失时，三个列表长度不一致，导致数据错乱。

提取豆瓣电影信息（稳健版）# 鲁棒性是很差的翻译

推荐做法：先找到每部电影对应的重复块（block），再在块内各自提取字段：

```
blocks = soup.select('#content li')

data = []
for block in blocks:
    # 提取英文标题（若无则为 None）
    element = block.select_one('.title:nth-child(2)')
    title_original = element.text[3:] if element else None

    data.append({
        "title_cn": block.select_one(".title:first-child").text,
        "title_original": title_original,
        "rating": block.select_one(".rating_num").text,
        "quote": block.select_one(".quote span").text,
        "url": block.select_one(".hd a").get("href")
    })
```

核心技巧：寻找页面中每个数据项对应的最小重复块（如 标签），在块内分别提取各字段，可彻底避免错位问题。

1.5 定位元素：SelectorGadget 与 DevTools

SelectorGadget（辅助工具）

SelectorGadget 是一个浏览器扩展，访问 <https://selectorgadget.com/> 安装。

使用方法：点击目标元素，工具会自动生成对应的 CSS 选择器字符串，可直接用于 BeautifulSoup。

启动它，页面右下角会弹出一个操作框。点击您希望匹配选择器的页面元素（该元素将变为绿色）。SelectorGadget 随后会为该元素生成一个最简 CSS 选择器，并将所有匹配该选择器的元素高亮显示（黄色）。现在点击高亮元素可将其从选择器中移除（变为红色），或点击未高亮元素将其添加到选择器。

SelectorGadget 给出的选择器可能不够精确，建议配合 DevTools 验证。

浏览器 DevTools (精确定位)

右键点击目标元素 → 选择「检查」/「Inspect」，即可在 Elements 面板中查看完整 HTML 结构。

在 Console 中验证选择器

在 DevTools 的 Console 面板中，可直接测试选择器是否生效：

```
// 选取所有匹配元素（返回数组，可查看元素数量）
$$('.item .hd a')

// 也可用标准 DOM API
document.querySelectorAll('.item .hd a')

// $0 表示当前在 Elements 面板中选中的元素
$0
```

1.6 翻页抓取 (Pagination)

豆瓣 Top 250 共 10 页，每页 25 条，URL 规律为 ?start=0, ?start=25 ... ?start=225。

利用循环实现自动翻页抓取：

程序员的哲学：凡重复操作超过 3 次，就应该用循环自动化！

将 HTML 表格直接转为 DataFrame

Pandas 提供了 `read_html()` 方法，可一步将 HTML 表格转换为 DataFrame：

```
import pandas as pd

# 因为要访问 wiki，使用代理穿透防火墙（proxy 是代理的意思）
# 127.0.0.1 是自己在操作的电脑 端口跟着软件走
# 大量爬的话我们需要租 ip 池子
proxies = {
    'http': '127.0.0.1:7890',
    'https': '127.0.0.1:7890'
}

url = 'https://en.wikipedia.org/wiki/Allergic_rhinitis'
response = requests.get(url, headers=headers, proxies=proxies)
html = response.text
soup = BeautifulSoup(html, 'html.parser')

# 定位目标表格
table_html = soup.select_one('.infobox')

# 转为 DataFrame（返回列表，取第一个）
tb = pd.read_html(str(table_html))[0]
```

第二部分：动态页面与 Selenium

2.1 静态页面 vs 动态页面

类型	特点	适用工具
静态页面	HTML 由服务器完整返回，requests.get() 获取全部数据	BeautifulSoup
动态页面	HTML 初始不完整，数据由 JavaScript 在用户交互时异步加载	Selenium

动态页面示例：Bilibili 首页

Bilibili 首页使用「无限滚动」加载更多视频，静态抓取只能获得初始的少量内容：

```
import requests
from bs4 import BeautifulSoup

url = 'https://www.bilibili.com/'
headers = {'User-agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)...' }

response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, 'html.parser')

# 只能获取初始渲染的少量视频标题
items = soup.select('.bili-video-card__info--tit a')
titles = [item.text for item in items] # 结果很少
```

2.2 Selenium：控制真实浏览器

Selenium 通过驱动真实浏览器执行 JavaScript，等待页面动态渲染完成后再提取数据。

代价：速度慢、资源占用多，适合无法用静态方式获取的页面。

#真开了一个浏览器！

示例一：滚动加载更多内容（Bilibili）

```
from selenium import webdriver
import time

driver = webdriver.Chrome()
driver.get('https://www.bilibili.com/')
time.sleep(2) # 等待初始加载

# 模拟滚动 5 次，触发动态加载
for i in range(5):
    driver.execute_script('window.scrollTo(0, document.body.scrollHeight);')
    time.sleep(1)

# 获取完整渲染后的 HTML
html = driver.page_source
soup = BeautifulSoup(html, 'html.parser')
items = soup.select('.bili-video-card__info--tit a')
titles = [item.text for item in items]
```

```
driver.quit() # 关闭浏览器
```

示例二：点击按钮加载更多（IT 之家博客）

```
driver = webdriver.Chrome()
driver.get('https://www.ithome.com/blog/')
time.sleep(2)

# 点击「加载更多」按钮 5 次
for i in range(5):
    button = driver.find_element('css selector', '.fl .more')
    button.click()
    time.sleep(1)

html = driver.page_source
soup = BeautifulSoup(html, 'html.parser')
items = soup.select('.c h2 a')
titles = [item.get('title') for item in items]

driver.quit()
```

Selenium 关键 API 速查

方法/属性	功能说明
driver.get(url)	打开指定 URL
driver.page_source	获取当前页面完整 HTML
driver.find_element('css selector', sel)	按 CSS 选择器查找单个元素
element.click()	点击元素
driver.execute_script(js)	执行 JavaScript 代码
time.sleep(n)	等待 n 秒（等待 JS 执行）
driver.quit()	关闭浏览器并释放资源

2.3 爬虫进阶工具

随着爬取需求的提升，可以考虑以下升级路径：

- **Scrapy**：Python 爬虫框架，支持异步并发、自动翻页、数据管道等，适合大规模抓取任务
- **Selenium / Playwright**：用于动态页面的浏览器自动化工具

2.4 爬虫伦理与合规

网络爬虫虽然技术上可行，但需要注意法律和伦理问题：

限制爬取速度

大量快速请求会给服务器造成额外负担，应当：

- 减少并发线程数量
- 在连续请求之间添加时间间隔（建议 3 秒以上）

```
import time
time.sleep(3) # 每次请求后等待 3 秒
```

检查 robots.txt

找到这个只要在链接后面加一个/robots.txt

每个网站都可以在根目录放置 robots.txt 文件声明爬取规则，例如：

www.douban.com/robots.txt

```
# robots.txt 示例内容
User-agent: *
Disallow: /login
Disallow: /subject_search
```

检查服务条款 (Terms of Service)

<https://app. /agreementGsw.aspx> 用户协议

<https://app. n/yinsilos.aspx> 隐私政策

- 爬取的数据能否合法使用或再分发？这是法律灰色地带
- 爬取公开数据通常被视为合法，但商业用途需格外谨慎
- 如有疑问，建议直接联系网站运营方获取书面授权

网络爬虫本身处于法律灰色地带。请务必遵守 robots.txt 规则、控制爬取速度，不要用于商业目的或大规模数据分发。

第三部分：应用程序接口 (API)

3.1 什么是 API

API (Application Programming Interface, 应用程序编程接口) 是不同软件程序之间通信的约定，本质上是一种「服务合同」。

爬虫 vs API 对比

方式	优点	缺点
网络爬虫	适用范围广，几乎所有网站都能爬	数据凌乱，需清洗，可能违反条款
API	数据干净（通常为 JSON），官方授权	访问受限，可能需要付费或申请权限

当目标网站提供 API 时，优先使用 API，这样获取的数据更干净且合法合规。

3.2 如何学习并使用任何 API

4. 查找官方文档（官网或 docs 页面）
5. 找到以下关键信息：端点（Endpoint）—— 特定功能对应的 URL；请求参数（Parameters）；响应格式（通常为 JSON）
6. 用小请求测试验证
7. 迭代优化，处理分页、错误等情况

3.3 实战示例：GitHub API

官方文档：<https://docs.github.com/en/rest/search/search>

基本请求

```
import requests

url = 'https://api.github.com/search/repositories'
params = {
    "q": "machine learning", # 搜索关键词
    "sort": "stars"          # 按 Star 数排序
}
# 以上参数自动拼接到 URL: ?q=machine+learning&sort=stars

response = requests.get(url, params=params)
data = response.json() # 返回 Python 字典
```

理解响应结构

```
# data 的结构示例
data = {
    "total_count": 381250,
    "incomplete_results": False,
    "items": [
        {
            "name": "tensorflow",
            "description": "An Open Source ML Framework",
            "stargazers_count": 185000,
            "html_url": "https://github.com/tensorflow/tensorflow"
        },
        # ... 更多仓库
    ]
}
```

3.4 API 分页处理

API 通常不会一次返回所有数据，而是分页返回。需要循环请求多页：

```
import pandas as pd

url = 'https://api.github.com/search/repositories'
data = []

for page in range(4): # 获取前 4 页
    params = {
        "q": "machine learning",
        "sort": "stars",
        "per_page": 30, # 每页 30 条
        "page": page # 当前页码
    }
    response = requests.get(url, params=params)
    results = response.json()

    for repo in results['items']:
        data.append({
            "name": repo["name"],
            "description": repo["description"],
            "stars": repo["stargazers_count"],
            "url": repo["html_url"]
        })

# 转为 DataFrame
df = pd.DataFrame(data)
print(df.head())
```

GitHub API 对未认证请求有速率限制（60 次/小时）。如需更高频率，可在请求头中加入 Personal Access Token。

```
# 添加认证 Token
headers = {
    "Authorization": "token YOUR_GITHUB_TOKEN",
    "Accept": "application/vnd.github+json"
}
response = requests.get(url, params=params, headers=headers)
```

课程总结

通过本课程，掌握了从互联网构建数据集的两大技术路径：

技术	核心工具	适用场景	关键注意事项
静态爬虫	requests + BeautifulSoup	HTML 完整返回的页面	设置 User-Agent，处理翻页
动态爬虫	Selenium	JavaScript 渲染的动态页面	速度慢，需 time.sleep() 等待
API 调用	requests + .json()	有官方 API 的数据源	处理分页、认证、速率限制

核心要点回顾

- Web 爬虫 = 下载 HTML + 解析结构 + 提取数据
- 用 User-Agent 伪装浏览器，避免被 418 拒绝
- CSS 选择器是定位元素的核心语法
- 用重复结构块（block-level）提取数据，避免错位
- 动态页面需要 Selenium 控制浏览器执行 JS
- API 返回干净的 JSON 数据，是更优先的选择
- 务必遵守 robots.txt、控制频率、注意法律合规

附录：CSS 选择器完整参考

选择器	示例	匹配说明
元素名	div	所有 <div> 元素
类选择器	.title	所有 class 含 title 的元素
ID 选择器	#content	id=content 的元素（唯一）
后代选择器	.quote span	.quote 内所有 后代
子选择器	div > p	div 的直接子 <p>
相邻选择器	div + p	紧接在 div 后的 <p>
多类选择器	.title.playable	同时有 title 和 playable 类
属性选择器	[class="bd"]	class 属性等于 bd
第 n 个子节点	.title:nth-child(1)	有 title 类且是父节点第 1 个子节点
第一个子节点	.title:first-child	有 title 类且是父节点第一个子节点
同类第 n 个	.item:nth-of-type(2)	.item 类中的第 2 个同类元素